

SYSTEM DESIGN — LEARNING ROADMAP

V1

20 branches, six themes, ordered so each one builds on the last
foundations · data · distributed core · traffic & resilience · integration · cross-cutting

WHAT SYSTEM DESIGN IS, AND WHY LEARN IT

start here — the definition, why it matters, and how to use this sheet

THE DEFINITION

System design is deciding how the parts of a software system fit together so that it meets its requirements **at the scale, latency and reliability it actually needs** — and keeps meeting them as those change.

It is the layer above writing code and below business strategy: what components exist, how they communicate, where state lives, what happens when a part fails, and what it costs.

The distinguishing skill is not knowing components. It is choosing between them under constraints you were given, and being able to say what each choice costs. Anyone can name a cache; the work is knowing when adding one makes things worse.

WHAT A DESIGN HAS TO ANSWER

- What are the functional and non-functional requirements, stated in numbers?
- What is the data model, and what are the access patterns?
- Where is the bottleneck, and what happens when it is reached?
- What fails, how is it detected, and what does the user see?
- What does it cost, and how does that scale?
- What would you revisit, and on what signal?

WHY LEARN IT

- It is what seniority is measured by.** Beyond a certain level, the question stops being “can you implement this?” and becomes “should we build it this way?”
- The expensive mistakes are architectural.** A slow function is an afternoon; the wrong partition key, the wrong consistency model or the wrong service boundary is a year.
- It is the common language** between engineering, product and operations for discussing trade-offs.
- It transfers.** Frameworks and cloud services change constantly; queuing, consistency, caching and failure behaviour do not.
- It is the interview.** For most senior roles the system design round carries more weight than the coding one.

The most common way people study this badly is memorising reference architectures for well-known products. Interviewers and real systems both change one constraint and watch what happens. Learn the reasoning, and the architectures follow; learn the architectures, and nothing follows.

THE HABIT WORTH BUILDING FIRST

For every technique on this sheet, learn it as a **pair**: what it buys and what it costs. Sharding buys write throughput and costs joins and transactions. Caching buys latency and costs correctness. Microservices buy independent deployment and cost you a distributed system to debug.

WHAT THIS SHEET IS, AND IS NOT

It is	It is not
A map of the territory, with what to learn in what order	A tutorial — each branch is a subject, not a lesson
A checklist for finding your own gaps	A list of answers to memorise
Vendor-neutral concepts	Tied to one cloud or stack
Ordered so each theme builds on the last	A strict sequence — themes B to F overlap in practice

HOW LONG THIS TAKES

Theme A is a few weeks of focused study. The rest is measured in years, because most of it only lands once you have operated something that failed in that particular way. That is normal and not a reason to defer starting.

WHERE TO BEGIN IF YOU ARE UNSURE. TAKE ONE SYSTEM YOU ALREADY WORK ON. DRAW IT. THEN ANSWER THE SIX QUESTIONS ON THE LEFT ABOUT IT. THE BRANCHES WHERE YOU CANNOT ANSWER ARE YOUR READING LIST, IN PRIORITY ORDER — WHICH IS MORE USEFUL THAN ANY GENERIC SEQUENCE.

A note on the interview framing. These topics are heavily associated with hiring, and that is a legitimate reason to learn them. But the reason they are asked is that they predict the work — designing under constraint, and knowing what you gave up.

THE MAP

learn left to right within a theme; themes can be taken in any order once the foundations are in place

SYSTEM DESIGN

Learning Roadmap · 20 branches
every branch below is expanded in a card on this sheet

A · FOUNDATIONS

- Fundamentals & goals
- Requirements & estimation
- Networking & protocols
- API design

B · DATA

- Data stores by family
- Storage engines & indexing
- Caching

C · DISTRIBUTED CORE

- Distributed fundamentals
- Consensus & coordination
- Scaling & distribution
- Partitioning & consistent hashing

D · TRAFFIC & RESILIENCE

- Load balancing, proxies, gateways
- Resilience patterns
- Architectural styles

E · INTEGRATION

- Distributed transactions
- Messaging & event streaming

F · CROSS-CUTTING

- IAM, authN & authZ
- Observability & operations
- Algorithms for system design
- Delivery, DR & cost

HOW TO USE THIS ROADMAP

- Learn theme A end to end first — everything after it assumes you can size a system and choose a protocol.
- Take B and C together. Data models and distribution constraints only make sense against each other.
- Build something small at each step. A branch you have not implemented is a branch you have read, not learned.
- Then D, E and F, which are mostly about keeping a working system working under load and failure.

5 · Solve full design problems only after A–C. Doing them first teaches you the answers to specific questions rather than the reasoning.

THE HABIT THAT MATTERS MORE THAN THE LIST

Every item here is a trade-off, not a best practice. Sharding buys write throughput and costs you joins and transactions. Caching buys latency and costs you correctness. Microservices buy independent deployment and cost you a distributed system to debug.

Learn each topic as a pair: what it buys, and what it costs. A candidate who names the cost is doing system design; one who names only the benefit is reciting. The same is true on the job, where the cost arrives about six months after the benefit.

Nothing on this sheet is free, and no item is correct in every context — which is why the reasoning transfers and the list does not.

1 FUNDAMENTALS & GOALS

THE FIVE PROPERTIES

- Scalability** — handles growth in load, data or users
- Availability** — fraction of time it serves requests
- Reliability** — serves them correctly; not the same thing
- Performance** — latency and throughput under load
- Maintainability** — the one that decides your velocity in year two

AVAILABILITY, AS A NUMBER

- 99% = 3.65 days of downtime a year
- 99.9% = 8.8 hours · 99.99% = 53 minutes
- 99.999% = 5 minutes — very expensive, rarely required
- Serial dependencies **multiply**: five 99.9% services in a chain give = 99.5%

CORE READING

- Latency vs throughput vs concurrency
- Little’s Law — $L = \lambda W$
- Why adding servers stops helping (contention and coherency)

Availability + reliability. A service that returns wrong answers quickly is highly available and completely unreliable. Interviews and incident reviews both punish conflating them.

2 REQUIREMENTS & ESTIMATION

CLARIFY BEFORE DESIGNING

- Functional requirements — what it must do
- Non-functional — scale, latency target, consistency, availability, retention, budget
- Explicit out-of-scope, so the design has an edge
- Who the users are, and the read/write ratio

BACK-OF-THE-ENVELOPE

- DAU / MAU — requests per second, peak vs average
- Peak is typically several times the mean — design for peak
- Storage per record × records × replication × retention
- Bandwidth, connection count, memory for cache working set
- Powers of two, and rough latency numbers (memory vs SSD vs network vs cross-region)
- PERCENTILES, NOT AVERAGES**
- p50, p95, p99, p99.9 — the tail is what users feel
- Fan-out amplifies tails: 100 parallel calls at p99 = 1s means most requests hit one
- Averages hide bimodal behaviour completely

STATE ASSUMPTIONS OUT LOUD AND WRITE THEM DOWN. An estimate with a stated assumption is engineering; a number with no assumption is a guess that someone will later treat as a requirement.

3 NETWORKING & PROTOCOLS

FOUNDATIONS

- TCP vs UDP — ordering and delivery vs latency
- DNS resolution, TTL, and why deploys wait on it
- TLS handshake, conn, session resumption, mTLS
- Latency budget: RTT, bandwidth-delay product, geography

HTTP VERSIONS

- HTTP/1.1** — one request per connection at a time, head-of-line blocking
- HTTP/2** — multiplexed streams over one TCP connection; still TCP-level HOL blocking
- HTTP/3 / QUIC** — over UDP; removes transport HOL blocking, faster connection setup

REAL-TIME AND STREAMING

- Short polling — long polling — SSE — WebSocket
- Pick by direction: SSE is server-client only; WebSocket is bidirectional
- Connection state is the cost — WebSockets make your servers stateful
- gRPC / HTTP/2 for internal service-to-service

Absent from most roadmaps, and asked constantly. “How does the client learn about a new message?” is a protocol question, and polling, SSE and WebSockets give three different systems.

4 API DESIGN

STYLES, AND WHEN EACH FITS

- REST** — resource-oriented, cacheable, ubiquitous. The default for public APIs
- GraphQL** — client picks the shape; solves over-fetching, costs you caching and query-cost control
- gRPC** — binary, typed contracts, streaming. Strong for internal calls, awkward from browsers
- Webhooks** — server-to-server push; needs retries, signing and idempotency

CONTRACT CONCERNS

- Versioning — path, header, or never break; pick one and hold it
- Pagination — cursor over offset at scale
- Idempotency keys on every unsafe retrievable operation
- Consistent error envelope and error codes
- Rate limit headers, and what a 429 should tell the caller
- Filtering, sorting, partial responses, bulk endpoints

Backward compatibility is the whole discipline. Adding a field is safe; removing one, or renaming one, or tightening validation is a breaking change even when the compiler stays quiet.

5 DATA STORES BY FAMILY

FAMILIES AND REPRESENTATIVES

- Relational** — PostgreSQL, MySQL, Oracle, SQL Server
- Document** — MongoDB, Couchbase, DocumentDB
- Key-value** — Redis, DynamoDB, Riak
- Wide-column** — Cassandra, ScyllaDB, HBase, Bigtable
- Columnar / analytics** — ClickHouse, Snowflake, BigQuery, Druid
- Graph** — Neo4j, Amazon Neptune, JanusGraph
- Time series** — InfluxDB, TimescaleDB, Prometheus
- Search** — Elasticsearch, OpenSearch, Solr
- NewSQL / distributed SQL** — CockroachDB, TiDB, YugabyteDB, Spanner
- Vector** — pgvector, Pinecone, Weaviate, Milvus, Qdrant
- Object storage** — S3, GCS, Azure Blob
- Embedded** — SQLite, DuckDB, RocksDB

VECTOR STORES ARE NOW A STANDARD FAMILY, NOT A NICHE. ANY SYSTEM WITH SEMANTIC SEARCH, RECOMMENDATIONS OR RETRIEVAL-AUGMENTED GENERATION NEEDS ONE — AND VECTORSTORE OFTEN REMOVES THE NEED FOR A SEPARATE SERVICE ENTIRELY.

Start relational. Postgres handles documents, key-value, full-text, geospatial and vectors competently. Adding a second store adds consistency, operations and failure modes — earn it with a measured reason, not an anticipated one.

6 STORAGE ENGINES & INDEXING

THE TWO ENGINE FAMILIES

- B-tree** — update in place. Predictable reads, slower random writes. Postgres, MySQL/InnoDB
- LSM tree** — append to a memtable, flush to sorted files, compact in background. Fast writes, read amplification, compaction cost. Cassandra, RocksDB, ScyllaDB

Read / write / space amplification — the three-way trade every engine picks a point on

INDEXING

- Primary vs secondary; clustered vs non-clustered
- Composite indexes and why **column order decides usability**
- Covering indexes — answer from the index, never touch the row
- Partial and functional indexes
- Inverted index for search; geospatial and vector indexes
- Every index speeds reads and slows every write

This branch explains the rest of the data section. “Why is Cassandra test at writes?” and “why does this query need a full scan?” are both storage-engine questions, and neither is answerable from a list of database names.

7 CACHING

PATTERNS

- Cache-aside** — app reads cache, misses, loads, populates. The default
- Read-through** — cache loads on miss itself
- Write-through** — write both, consistent, slower writes
- Write-behind** — write cache, flush async. Fast, risks loss
- Refresh-ahead** — refresh hot keys before expiry

EVICTON & INVALIDATION

- LRU, LFU, FIFO, TTL, and random
- Invalidation is the hard half — TTL, explicit, or event-driven
- Versioned keys avoid deletion races

FAILURE MODES

- Stampede / thundering herd** — many misses at once on expiry. Fix with jittered TTL, locks or early recompute
- Hot key** — one key saturates one node
- Penetration** — repeated misses for keys that do not exist; cache the negative
- Cold start** after a restart or deploy

WHERE CACHES LIVE

- Browser → CDN → gateway → application → distributed cache → database buffer pool

Deserves a full branch, not a footnote. Caching is the most commonly deployed and most commonly mis-configured technique in the whole roadmap — and every entry above is a real production incident class.

8 DISTRIBUTED FUNDAMENTALS

THEORY YOU WILL ACTUALLY USE

- CAP** — under a partition, choose consistency or availability. Not a permanent choice of two from three
- PACELC** — and when there is no partition, latency vs consistency. The more useful framing day to day
- ACID vs BASE**
- Consistency models — strong, sequential, causal, eventual, read-your-writes, monotonic reads

WHAT ACTUALLY GOES WRONG

- Partial failure — the defining property of a distributed system
- Network is unreliable, latency non-zero, topology changes — the eight fallacies
- Timeouts, retries, and the retry storms they cause
- Idempotency — the only thing that makes retries safe
- Exactly-once delivery does not exist; at-least-once + idempotency does
- Clock skew — never order events by wall clock across machines
- Replication lag, stale reads, split-brain
- Backpressure and head-of-line blocking

The single most useful sentence here: a timeout tells you nothing about whether the other side did the work. Every correctness argument in a distributed system starts from that.

9 CONSENSUS & COORDINATION

AGREEMENT

- Raft** — leader election, log replication, safety. Learn this one properly
- Paxos / Multi-Paxos** — the ancestor; know the shape
- ZAB** (ZooKeeper), and where etcd and Consul fit
- Why consensus needs a majority, and why even-sized clusters are a mistake

QUORUMS

- $R + W = N$ gives read-your-writes on a quorum store
- Tunable consistency in Cassandra and DynamoDB

HINTED HANDOFF, READ REPAIR, ANTI-ENTROPY

- ORDERING & COORDINATION
- Lamport clocks, vector clocks, version vectors
- Leader election and fencing tokens
- Distributed locks — and why lock + timeout is not mutual exclusion
- Failure detection: heartbeats, gossip, phi-accrual
- CRDTs for conflict-free merges

A conspicuous gap in most roadmaps. “How does the cluster agree on a leader?” and “what happens when the leader is only partially unreachable?” are the questions that separate familiarity from understanding.

10 SCALING & DISTRIBUTION

DIRECTIONS

- Vertical — simple, bounded, no distribution problems. Do this first
- Horizontal — unbounded, and buys you every problem in theme C
- Stateless services scale trivially; state is the actual problem

REPLICATION

- Leader-follower** (formerly master-slave) — one writer, many readers
- Multi-leader** — write anywhere, conflict resolution required
- Leaderless** — quorum reads and writes
- Synchronous vs asynchronous, and the replication lag it creates
- Follower, promotion, and split-brain protection

OTHER LEVERS

- Read replicas, CQRS-style read models, denormalisation
- CDN and edge for static and cacheable content
- Autoscaling — and why it never saves you from a cold cache
- Multi-region: active-passive vs active-active, data residency
- Cell-based architecture and shuffle sharding to bound blast radius

Prefer leader-follower / primary-replica terminology. It is both the industry standard now and more precise about what the roles do.

11 PARTITIONING & CONSISTENT HASHING

PARTITIONING STRATEGIES

- Range** — ordered scans work; hotspots on sequential keys
- Hash** — even spread; range queries stop working
- Directory** — flexible, adds a lookup and a dependency
- Composite keys: partition key + sort key

CONSISTENT HASHING

- The hash ring, and why plain hash % N mangles almost everything on resize
- Virtual nodes** — many points per physical node, for even distribution and smooth rebalancing
- Adding and removing a node moves only that node’s share
- Used by Cassandra, DynamoDB, Memcached clients, and load balancers for sticky routing
- Rendezvous hashing as a simpler alternative

WHAT GOES WRONG

- Hot partition / celebrity key** — one key or tenant dominates. Salt it, or split it
- Skew from a poorly chosen partition key is the most common scaling defect
- Resharding a live system is difficult — choose the key as if you cannot change it
- Cross-partition joins and transactions are what you gave up

12 LOAD BALANCING, PROXIES & GATEWAYS

THREE ROLES, OFTEN CONFUSED

- Reverse proxy** — sits in front of servers: TLS termination, compression, caching, static content. NGINX, HAProxy, Traefik, Envoy
- API gateway** — a reverse proxy plus API concerns: authN/authZ, rate limiting, quotas, request transformation, protocol translation, aggregation, per-consumer metering. Kong, Apigee, AWS API Gateway, Tyk
- Service mesh** — proxies beside every service for east-west traffic: mTLS, retries, circuit breaking, traffic splitting, telemetry. Istio, Linkerd
- Forward proxy** — sits in front of clients; controls egress

LOAD BALANCING

- L4** — transport level, fast, protocol-blind. L7 — understands HTTP: can route on path, header, cookie
- Algorithms: round robin, weighted, least connections, least response time, consistent hashing
- Health checks, connection draining, sticky sessions and why to avoid them
- Global load balancing: DNS, anycast, GeoDNS

NORTH-SOUTH VS EAST-WEST. GATEWAYS HANDLE TRAFFIC ENTERING THE SYSTEM; A MESH HANDLES TRAFFIC BETWEEN SERVICES. CHOOSING BETWEEN THEM IS A CONFIGURATION ERROR — MOST LARGE SYSTEMS RUN BOTH.

13 RESILIENCE PATTERNS

STOP A FAILURE SPREADING

- Timeout** — bound every remote call, always. Nothing else works without it
- Retry with exponential backoff + jitter** — jitter is not optional; without it every client retries in lockstep
- Circuit breaker** — stop calling a dependency that is already failing
- Bulkhead** — isolate pools so one dependency cannot exhaust all threads or connections
- Rate limiting & throttling** — protect the callee
- Load shedding** — reject early under overload rather than degrading for everyone
- Graceful degradation** — serve less rather than nothing
- Fallback** — cached, default, or queued; never a second remote call
- Idempotency** — what makes every retry above safe
- Health checks** — liveness must not fail because a dependency is down

These are not GoF design patterns and should not be listed beside Singleton and Factory. Those describe object construction and collaboration inside one process; these describe how a system behaves when a remote dependency fails. Different problem, different scale, different chapter.

14 ARCHITECTURAL STYLES

SYSTEM-LEVEL STYLES

- Monolith** — one deployable. Simplest, and correct far more often than its reputation suggests
- Modular monolith** — enforced internal boundaries, one deployment. Usually the right next step
- Microservices** — independent deployment and scaling, at the cost of a distributed system
- Service-oriented, event-driven, serverless**
- Layered, hexagonal / ports and adapters, clean architecture**
- CQRS** — separate read and write models
- Pipes and filters, actor model, cell-based**
- SUPPORTING PATTERNS**
- BFF** — a backend per client type
- Sidecar / ambassador**
- Strangler fig** — the standard way to replace a legacy system incrementally
- API composition and service discovery**
- Database per service, and the joins you lose

Conway’s Law is a design constraint, not a joke. Systems reflect the communication structure of the teams that build them, so a service boundary that cuts across a team boundary will be fought continuously until one of them moves.

15 DISTRIBUTED TRANSACTIONS

LOCAL FIRST

- Isolation levels: read uncommitted → read committed → repeatable read → serializable
- The anomalies each one permits: dirty read, non-repeatable read, phantom, write skew
- Optimistic (version column) vs pessimistic (locking) concurrency
- MVCC — how most modern databases avoid readers blocking writers

ACROSS SERVICES

- 2PC** — atomic, blocking, and a coordinator you must keep alive. Rare in modern designs
- 3PC** — reduces blocking, adds complexity, still assumes a lot
- Saga** — a sequence of local transactions with compensating actions. Choreography (events) or orchestration (a coordinator)
- TCOC** — try, confirm, cancel with explicit reservation
- Outbox** — write state and event in one local transaction; publish from the outbox
- Inbox / idempotent consumer** — deduplicate on receipt
- CDC** — read the log rather than dual-writing

Sagas give up isolation, not just atomicity. Intermediate states are visible to other readers, so “the order exists but is unpaid” must be a state your domain model actually has — not a window you hope nobody looks into.

16 MESSAGING & EVENT STREAMING

QUEUE VS LOG — THE DISTINCTION THAT MATTERS

- Queue** (RabbitMQ, SQS, ActiveMQ) — a message is delivered, acknowledged and **removed**. Competing consumers share the work. Per-message acks, visibility timeouts, DLQs
- Log** (Kafka, Pulsar) — records are appended and **retained**; consumers track an offset. Many independent consumers read the same data, and replay is a normal operation
- Choosing wrongly here shapes everything downstream

MECHANICS

- Topics, partitions, consumer groups, offsets
- Ordering is per partition, never global
- At-most-once / at-least-once / effectively-once
- DLQ, poison messages, retry topics
- Backpressure, lag monitoring, rebalancing
- Fan-out, competing consumers, request-reply
- EVENT-DRIVEN DESIGN
- Event sourcing, event replay, materialized views
- Schema registry and versioning (Avro, Protobuf) — events are a contract
- Event-carried state transfer vs thin events
- Stream processing: windowing, watermarks, joins — Flink, Kafka Streams

17 IAM, AUTHN & AUTHZ

TWO DIFFERENT QUESTIONS

- Authentication** — who is this? **Authorization** — what may they do? Never one control

PROTOCOLS & TOKENS

- OAuth 2.0 / O / Z / I** — delegated authorization; grant types and which are still safe
- OIDC** — authentication layered on OAuth ID tokens
- SAML** — enterprise federation, still everywhere
- JWT** — structure, signing, validation, expiry, and why revocation is hard
- Session cookies vs bearer tokens; refresh token rotation
- Token exchange for on-behalf-of calls between services
- API keys and HMAC request signing

AUTHORIZATION MODELS

- RBAC roles** · **ABAC attributes** · **ReBAC** relationships (Zanzibar-style)
- Permission-based checks, not role-name checks
- Multi-tenancy and tenant isolation tests

SERVICE-TO-SERVICE & OPERATIONS

- mTLS and workload identity — network position is not authentication
- Secrets management and rotation (Vault, cloud KMS)
- MFA, account recovery, breach response
- Providers: Keycloak, Auth0, Okta, Cognito, Microsoft Entra ID, Google Identity

18 OBSERVABILITY & OPERATIONS

THE THREE SIGNALS

- Logs** — structured, correlated, sampled. ELK, Loki
- Metrics** — cheap, aggregatable, low cardinality. Prometheus, Grafana
- Traces** — one request across services. Jaeger, Tempo, OpenTelemetry
- Correlation and trace IDs propagated through every hop, including async ones
- OpenTelemetry as the vendor-neutral instrumentation standard

KNOWING WHAT TO MEASURE

-